

Experiment 7: Case Study on Incremental Learning

Implementation of Incremental Intrusion Detection for Real-Time Network Traffic using Stochastic Gradient Descent (SGD)

Aim

To implement an incremental learning model using Stochastic Gradient Descent that continuously updates itself as new network traffic records arrive, and to evaluate its accuracy on unseen data.

Case Study

Modern organizations generate massive amounts of network traffic every second. Traditional machine learning models used in Intrusion Detection Systems (IDS) are trained on historical data and require complete retraining whenever new data arrives. This retraining process is time consuming, computationally expensive and unsuitable for a real-time cyber security environment.

To address this challenge, an incremental learning approach using Stochastic Gradient Descent (SGD) was implemented. The model continuously learns from incoming network traffic and updates itself whenever new data arrives, without retraining on the entire dataset.

Theory

Incremental learning, also called online learning, is a technique in which a model is updated using one record or a small batch of records at a time instead of the whole dataset. Scikit-learn provides the `partial_fit()` method for estimators that support this behaviour. The `SGDClassifier` updates its weights using the gradient computed from each arriving sample, so the memory requirement stays constant no matter how much data has been seen. When the loss is set to `log_loss`, the classifier behaves like logistic regression trained with stochastic gradient descent, which makes it suitable for binary classification such as separating normal traffic from an intrusion.

Requirements

- Python 3.x
- Jupyter Notebook
- NumPy
- Scikit-learn

Step 1: Import Libraries

```
import numpy as np
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import accuracy_score
```

Interpretation: Imports NumPy for data generation, the SGD classifier for incremental learning and the accuracy metric for evaluation.

Step 2: Generate Initial Data

```
np.random.seed(42)
X_train = np.random.randn(100, 2)
y_train = (X_train[:, 0] + X_train[:, 1] > 0).astype(int)
```

Interpretation: `X_train` creates 100 training samples with 2 numerical features each. `y_train` generates binary class labels.

Step 3: Incremental Learning Model

```
model = SGDClassifier(loss="log_loss")
```

Interpretation: Creates an SGD classifier for binary classification. The log_loss loss makes the model perform logistic regression using SGD optimization.

Step 4: Initial Training

```
model.partial_fit(  
    X_train,  
    y_train,  
    classes=np.array([0, 1])  
)
```

Interpretation: partial_fit() trains the SGD model on the initial training data and enables incremental learning by recognising classes 0 and 1.

Step 5: Simulation Training

```
for i in range(20):  
    X_new = np.random.randn(1, 2)  
    y_new = (X_new[:, 0] + X_new[:, 1] > 0).astype(int)  
    prediction = model.predict(X_new)  
    print(f"Record {i + 1}:")  
    print("Prediction:", prediction[0])  
    print("Actual:", y_new[0])  
    model.partial_fit(X_new, y_new)
```

Interpretation: Simulates 20 new incoming data records and predicts their class labels. After each prediction the model is updated incrementally using partial_fit() so that it learns from the new record.

Step 6: Test Accuracy

```
X_test = np.random.randn(50, 2)  
y_test = (X_test[:, 0] + X_test[:, 1] > 0).astype(int)  
prediction = model.predict(X_test)  
acc = accuracy_score(y_test, prediction)  
print("Accuracy:", acc)
```

Interpretation: The test accuracy is approximately 0.98, that is 98%.

Complete Program

```
import numpy as np  
from sklearn.linear_model import SGDClassifier  
from sklearn.metrics import accuracy_score  
  
np.random.seed(42)  
  
X_train = np.random.randn(100, 2)  
y_train = (X_train[:, 0] + X_train[:, 1] > 0).astype(int)  
  
model = SGDClassifier(loss="log_loss")  
  
model.partial_fit(X_train, y_train, classes=np.array([0, 1]))  
  
for i in range(20):  
    X_new = np.random.randn(1, 2)
```

```

y_new = (X_new[:, 0] + X_new[:, 1] > 0).astype(int)
prediction = model.predict(X_new)
print(f"Record {i + 1}:")
print("Prediction:", prediction[0])
print("Actual:", y_new[0])
model.partial_fit(X_new, y_new)

X_test = np.random.randn(50, 2)
y_test = (X_test[:, 0] + X_test[:, 1] > 0).astype(int)

prediction = model.predict(X_test)
acc = accuracy_score(y_test, prediction)
print("Accuracy:", acc)

```

Sample Input

No external input file is required. The data is generated internally:

```

Initial training : 100 samples, 2 features
Incoming stream  : 20 records, one at a time
Testing          : 50 samples
Class rule       :  $y = (x_1 + x_2 > 0)$ 

```

Expected Output

```

Record 1:
Prediction: 1
Actual: 1
Record 2:
Prediction: 0
Actual: 0
Record 3:
Prediction: 1
Actual: 1
...
Record 20:
Prediction: 0
Actual: 0

```

Accuracy: 0.98

The exact accuracy varies slightly between runs because the data is generated randomly, but it stays high, normally between 0.90 and 1.00.

Questions

Q1. Why are we using SGD and not a Decision Tree, Random Forest or Neural Network?

Decision Tree

- It does not support `partial_fit()`.
- When a new network record arrives the entire tree must be rebuilt.

Random Forest

- It consists of multiple decision trees and requires retraining many trees when new data arrives, making it computationally expensive.

Neural Network

- It can learn incrementally but requires more computation, more data and more parameter tuning

than SGD.

SGD

- It supports `partial_fit()` and learns from one record or a small batch at a time.
- It uses less memory.
- It is ideal for real-time streaming data.

Q2. Is an IDS (Intrusion Detection System) used in this case?

No, a complete IDS is not implemented in this study.

- The case study uses synthetic data instead of real network packets.
- There is no actual network monitoring or packet capture.
- The SGD model only simulates the detection component of an IDS by classifying data as normal or intrusion, and updating itself incrementally.

Result

The incremental intrusion detection model was successfully implemented using Stochastic Gradient Descent. The model was trained on the initial data, updated itself after every new incoming record using `partial_fit()`, and achieved a test accuracy of approximately 0.98, that is 98%. This confirms that incremental learning is suitable for real-time network traffic where retraining on the full dataset is not practical.